

Machine Learning Project Report

Dual-Pipeline Analysis: Regression & Classification

Introduction

This document outlines the complete machine learning pipeline implemented in **miniproject.ipynb**. The project features a dual-pipeline approach, demonstrating continuous regression on the Diamonds dataset and multinomial classification on the Water Potability dataset. It explains **what** was done, **where** it was implemented, and **why** specific methodologies were chosen.

Part 1: Continuous Regression Pipeline

Dataset: Diamonds

What is done:

Predicting the continuous numerical price of diamonds based on their physical characteristics.

Where is it done:

Steps 1.1 through 1.7 in miniproject.ipynb.

Why is it done:

To demonstrate linear modeling techniques and feature impact analysis on a continuous target variable.

Detailed Pipeline Execution:

- 1. Data Loading (Step 1.1):** Retrieved the dataset via kagglehub.
- 2. Preprocessing & Data Splitting (Step 1.2):** Separated features and target. Used One-Hot Encoding to convert categorical data ('cut', 'color', 'clarity') into numeric forms for model compatibility. Split data into 80% training and 20% testing sets to evaluate unseen performance. Applied StandardScaler to continuous features ensuring uniform gradient descent and preventing features with larger scales from dominating the distance metrics.
- 3. Model Training (Steps 1.3 - 1.5):** Trained three distinct linear models: Standard Linear Regression, Lasso Regression (L1 regularization to force sparsity and perform feature selection), and Ridge Regression (L2 regularization to prevent extreme weights and overfitting).
- 4. Evaluation & Visualization (Step 1.6):** Compared models using R^2 Score and Mean Squared Error (MSE), plotted with Seaborn to provide visual clarity on model performance.
- 5. Feature Importance (Step 1.7):** Extracted coefficients from the Lasso model to quantitatively demonstrate which physical features of a diamond most heavily dictate its price.

Part 2: Multinomial Classification Pipeline

Dataset: Water Potability

What is done:

Classifying water samples into three discrete potability categories based on their chemical attributes.

Where is it done:

Steps 2.1 through 2.11 in miniproject.ipynb.

Why is it done:

To implement robust preprocessing that prevents data leakage, resolve class imbalances using synthetic resampling, and evaluate advanced non-linear classification models.

Detailed Pipeline Execution:

- 1. Data Loading & Target Engineering (Step 2.1):** Downloaded data. Since the goal was multinomial classification, a synthetic 3-class target ('Potability_Class') was engineered based on pH quantiles.
- 2. Preprocessing & Anti-Leakage (Step 2.2):** The original 'ph' column was explicitly dropped to prevent severe data leakage. Handled missing data by imputing median values to maintain dataset integrity. Scaled the data with StandardScaler.
- 3. SMOTE Resampling (Step 2.2):** Applied the Synthetic Minority Over-sampling Technique (SMOTE) strictly on the training set. This was crucial to resolve class imbalance and ensure the models wouldn't become biased towards the majority class.
- 4. Model Initialization & Training (Steps 2.3 - 2.9):** Trained an array of algorithms ranging from simple to complex: Logistic Regression (L1 & L2 solvers), Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost.
- 5. Evaluation Matrices (Steps 2.4 - 2.9):** Calculated Accuracy, Precision, Recall, and Weighted F1-Scores. Visualized detailed performance breakdowns via Confusion Matrices.
- 6. Final Visual Comparison & Insights (Steps 2.10 - 2.11):** Compared all classifiers based on their F1-Scores on a seaborn bar plot. Finally, extracted feature importances via the Random Forest model to identify the most critical chemical components determining water safety.

Conclusion:

The pipeline demonstrates an end-to-end best-practice workflow for ML: proper data isolation, anti-leakage principles, rigorous feature scaling, imbalance resolution via SMOTE, and comprehensive metric evaluation across varying model complexities.